

Monitoring 47,000 distributed endpoints requires a decoupled, push-based telemetry architecture to prevent network bottlenecking and ensure reliable data ingestion. Attempting to centrally ping 47,000 schools will trigger ISP DDoS protections and yield inaccurate results due to central bandwidth limitations.

Here is the system architecture and implementation strategy for a national-scale DepEd internet monitoring dashboard.

1. Edge Telemetry (Data Collection)

The collection must happen at the school level and push outward to your central servers.

- **Lightweight Edge Agents:** Deploy a lightweight script (Python, Go, or a bash cron job) on existing infrastructure. This could run on the school's central MikroTik router, a dedicated Raspberry Pi, or a designated administrative PC that remains powered on.
-
- **Staggered Execution (Jitter):** Never schedule the speed test for the top of the hour (e.g., exactly 8:00 AM). If 47,000 schools run a speed test simultaneously, it will artificially congest national routing. Introduce randomized delays (jitter) so tests are distributed evenly across a 15-to-30-minute window.
-
- **Test Metrics:** The script should execute a standardized CLI test (like `iperf3` against regional servers or the Ookla Speedtest CLI) capturing Download, Upload, Latency (Ping), Jitter, and Packet Loss. It must also capture the local timestamp and the school's unique DepEd ID.
-

2. Ingestion and Storage Pipeline

The backend must handle high concurrency, as thousands of payloads will arrive simultaneously, especially when connectivity is restored after an outage.

- **API Gateway & Message Broker:** Funnel incoming JSON payloads through an API Gateway into a scalable message queue (like Google Cloud Pub/Sub or Apache Kafka). This decoupled design ensures that if your database goes down, telemetry data is queued and not lost.
-
- **Serverless Processing:** Use serverless functions to consume the message queue, validate the data (rejecting spoofed or malformed payloads), and enrich it with metadata (e.g., mapping the School ID to its Region, Division, and assigned ISP).
-
- **Time-Series Data Warehouse:** Store the enriched data in a time-series optimized database or data warehouse (such as BigQuery or TimescaleDB). Relational databases (like standard MySQL) will quickly degrade in query performance when aggregating millions of daily speed test rows.
-

3. The Hierarchical Dashboard

With 47,000 data points updating daily, a flat list is unusable. The visualization layer requires Role-Based Access Control (RBAC) to serve different operational needs.

- **National View (Central Office):** A map-based heatmap of the Philippines showing aggregate health by region. Include macro-metrics: total offline schools, average national bandwidth vs. contracted bandwidth, and ISP SLA (Service Level Agreement) compliance scores.
-
- **Division/Regional View:** Filtered views for Division Superintendents to identify localized outages, cluster failures (which might indicate a cut fiber line in a specific province), and underperforming districts.
-
- **School Level (Principals & IT):** Historical line charts showing daily performance, allowing local staff to prove to their ISP that bandwidth drops during school hours.
-
- **Tooling:** Use enterprise BI tools like Looker, Grafana (excellent for time-series networking data), or build a custom frontend using React/Next.js for granular control over the UI.
-

4. Operationalizing the Data

A dashboard is only useful if it drives action. Integrate automated alerting thresholds. If a school reports speeds 50% below their contracted rate for three consecutive days, or drops offline entirely, the system should automatically generate a trouble ticket. Furthermore, you can use the aggregated historical data to automatically calculate SLA penalties, demanding rebates from ISPs that fail to deliver the bandwidth paid for by the government.

DepEd Telemetry Pipeline

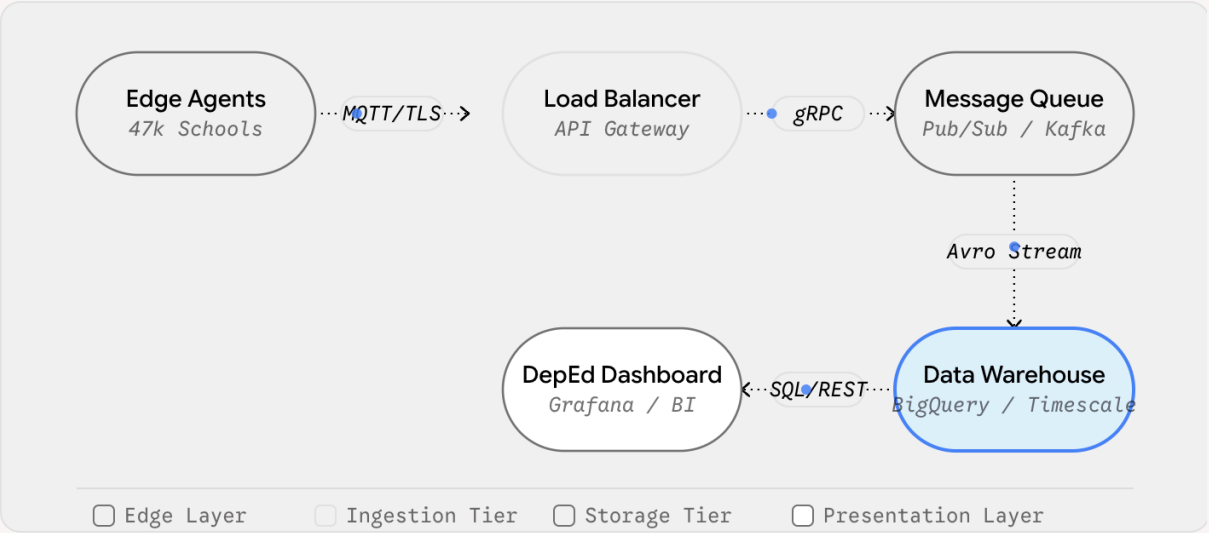
Edge-to-Dashboard Realtime Architecture (47,000 Schools)

● 47,120 Active Nodes • 68.5k msg/s (Peak)

Normal

Peak Traffic

Sync Mode



Data Warehouse (BigQuery / Timescale)

Storage Layer

COMMUNICATION PROTOCOL	SCALE & CAPACITY
JDBC / SQL / Columnar	12.4 TB historical metrics
OPERATIONAL ROLE	
Long-term telemetry persistence, time-series aggregation, analytical queries, rollup generation.	
FAULT TOLERANCE STRATEGY	
Automated point-in-time recovery with redundant multi-zone backups.	

Simulating 47,000 distributed endpoints requires moving beyond standard HTTP benchmarking to emulate actual behavioral patterns, network delays, and concurrent data ingestion at scale.

Here is how you can architect the simulation on Google Cloud Platform and integrate AI to validate your system.

Simulating the Telemetry Load

1. **Distributed Load Generation (Locust on GKE):** A single workstation will quickly hit TCP connection limits. Deploy a distributed load testing framework like Locust onto a Google Kubernetes Engine (GKE) cluster or across multiple Cloud Run instances. Locust allows you to write Python scripts that define complex, weighted behaviors.
- 2.
3. **Agent-Based Behavioral Modeling:** Treat each simulated school as an independent node in a high-concurrency multi-agent system. Instead of simply blasting requests, the script should emulate real-world router behavior: waking up, applying randomized jitter (e.g., waiting between 1 and 900 seconds to prevent thundering herd problems), authenticating, and pushing the JSON payload to Cloud Pub/Sub.
- 4.
5. **Simulated State Interruptions:** Program your agents to simulate outages. You can script scenarios where 5% of the "schools" suddenly drop connection, attempt exponential backoff retries, and push backlogged historical data once reconnected, testing how your BigQuery data warehouse handles out-of-order time-series data.
- 6.

How AI and Gemini Can Accelerate the Process

1. Synthetic Scenario Generation (Gemini)

Testing your dashboard with static dummy data (e.g., a flat 50 Mbps for every payload) will not validate your UI's ability to highlight problems. You can use Gemini via the Vertex AI API to programmatically generate thousands of realistic, highly varied JSON telemetry profiles. You can prompt the model to generate specific edge cases, such as:

- *"Generate 1,000 JSON speed test payloads for rural island schools using satellite internet, featuring high latency (600ms+), severe jitter, and intermittent packet loss."*
-
- *"Generate a sequential data profile for a school where bandwidth degrades specifically between 10:00 AM and 2:00 PM over a 5-day period."*
-

2. Predictive Anomaly Detection (Vertex AI)

Once your simulated data (and eventually, production data) is flowing into BigQuery, you can leverage Vertex AI to build machine learning models that understand the "normal" baseline for each specific school or region. Rather than relying on rigid, hard-coded alerts (e.g., "Alert if speed < 10 Mbps"), the AI can flag contextual anomalies—such as identifying when an entire cluster of schools drops offline simultaneously, indicating a cut regional fiber backbone rather than isolated hardware failures.

3. Automated Root Cause Summaries (Gemini)

When a school triggers a bandwidth alert, interpreting raw telemetry (packet loss, jitter, ping) can be difficult for non-technical administrative staff. You can pipe the anomalous time-series data and recent network logs directly into Gemini to generate a plain-English diagnostic summary. The AI can translate the raw metrics into actionable advice, such as: *"The connection is active, but high packet loss indicates physical line degradation; advise contacting the ISP to check the local fiber termination."*